

Campus Event Manager

Rapport de projet - Base de données NoSQL (MongoDB)

Auteur	Abidhia Fahmed
Cohorte	MCS DE1 - Master 1 Data Engineering and Cloud Computing
Dépôt GitHub	campus-event-manager
Base de données	MongoDB (campus_events)
Stack	Python / Flask / PyMongo

1. Présentation du projet

Ce projet remplace l'examen final du module NoSQL. Le sujet impose de concevoir une base de données documentaire depuis zéro et de construire une petite application qui s'appuie dessus pour de vrai, sans se contenter de scripts Mongo isolés. J'ai choisi le scénario proposé tel quel : une université qui n'a aucun outil pour gérer ses événements de campus (ateliers, confs, hackathons, meetups) et qui a besoin d'une application légère, «Campus Event Manager», pour centraliser les événements, les utilisateurs et les inscriptions.

Côté technique, j'ai opté pour Python avec Flask et PyMongo plutôt que Node ou Java. Deux raisons à ça : Python était déjà installé sur ma machine (pas besoin d'installer Node), et surtout PyMongo permet d'écrire les requêtes MongoDB directement, sans couche ODM qui les cache. Comme l'énoncé insiste sur le fait que les requêtes et pipelines doivent rester identifiables dans le code, ça m'a semblé plus cohérent de ne pas utiliser un outil comme Mongoose ou un ORM. MongoDB tourne en local sur mon Mac (installé via Homebrew), et l'interface est en Bootstrap avec des templates Jinja2 côté serveur - rien de compliqué, l'énoncé le précise bien, ce n'est pas un projet de design.

2. Modèle de données NoSQL

La base s'appelle `campus_events` et contient deux collections principales, `users` et `events`. Je n'ai pas ajouté de troisième collection : avec seulement deux entités vraiment distinctes (les personnes et les événements), une collection supplémentaire aurait été artificielle.

Collection `users`

Chaque document contient `_id`, `firstName`, `lastName`, `email`, `department`, `role`, `interests` (tableau de chaînes) et `createdAt`. Rien d'embarqué ici : un utilisateur est une entité simple et autonome.

Collection `events`

C'est la collection la plus intéressante niveau modélisation. Un document ressemble à ça (version raccourcie) :

```
{
  title: "Student Robotics Meetup",
  category: "Meetup", tags: ["robotics", "opensource"],
  startDate: ISODate(...), endDate: ISODate(...), capacity: 10,
  location: { building: "Engineering Building", room: "Workshop 3", campus: "Paris" },
  organizerId: ObjectId("..."),
  registrations: [
    { userId: ObjectId("..."), registeredAt: ISODate(...), status: "confirmed" },
    ...
  ]
}
```

```
]
}
```

3. Choix de modélisation : embedding vs référence

L'énoncé demande explicitement de justifier au moins deux décisions, donc voici les deux qui m'ont semblé les plus significatives.

location est embarquée. Le lieu d'un événement n'a aucun sens en dehors de l'événement lui-même : on ne va jamais chercher «tous les documents dans le bâtiment Innovation Center» comme une entité à part, et il n'est partagé par personne d'autre. Le mettre en objet embarqué évite une jointure inutile pour un objet qui ne contient que trois champs.

registrations est un tableau de sous-documents, mais userId à l'intérieur reste une référence. C'est le choix le plus important du modèle. J'ai embarqué les inscriptions directement dans l'événement parce que la page de détail d'un événement a besoin d'afficher et de modifier la liste des participants en permanence (compter les confirmés, vérifier qu'un utilisateur n'est pas déjà inscrit, ajouter/retirer une inscription) et le volume reste raisonnable - quelques dizaines de participants par événement au grand maximum, donc pas de risque de dépasser la taille limite d'un document. En revanche, à l'intérieur de chaque inscription, je ne recopie pas le nom ou l'email de l'utilisateur : je garde seulement son userId en référence. Un utilisateur existe indépendamment des événements (il peut changer d'email, être organisateur d'un événement et participant à dix autres), donc dupliquer ses infos dans chaque inscription aurait créé une incohérence à gérer dès qu'un profil change. C'est le même raisonnement pour organizerId sur l'événement : référence, pas embedding.

4. Architecture de l'application

Le dépôt suit à peu près la structure suggérée par l'énoncé :

```
database/init.py  # collections, validation, index
database/seed.py  # jeu de données reproductible
src/db.py         # connexion MongoClient (via .env)
src/app.py        # factory Flask, enregistre les 4 blueprints
src/routes/       # dashboard.py, events.py, users.py, analytics.py
src/templates/    # une vue Jinja2 par page
src/static/       # feuille de style complémentaire à Bootstrap
```

La connexion à MongoDB passe par `src/db.py`, qui lit `MONGO_URI` et `MONGO_DB_NAME` depuis un fichier `.env` (non versionné, avec un `.env.example` à côté pour montrer le format attendu). Chaque page correspond à un blueprint Flask séparé, ce qui fait que toutes les requêtes MongoDB liées à une page sont regroupées dans un seul fichier facile à retrouver plutôt que dispersées dans une couche DAO générique.

5. Pages de l'application

Dashboard

La page d'accueil affiche quatre compteurs (utilisateurs, événements, événements à venir, total des inscriptions), les 5 prochains événements et l'événement le plus populaire. Rien n'est figé en dur : le nombre d'utilisateurs et d'événements vient de count_documents, le total des inscriptions d'un pipeline \$unwind + \$count, et l'événement populaire d'un \$filter + \$size sur le tableau registrations pour ne compter que les statuts confirmed.

Events

Catalogue complet avec recherche par titre, filtre par catégorie, par tag, par période (passé/à venir), tri par date, et CRUD complet (création, édition, suppression). Un badge «Full» apparaît dès que le nombre d'inscriptions confirmées atteint la capacité. Le formulaire gère l'objet location embarqué et le tableau tags directement.

Détail d'un événement et inscriptions

C'est la page où se joue la gestion des inscriptions : choisir un utilisateur, l'inscrire (avec \$addToSet pour éviter tout doublon), refuser une inscription confirmée si la capacité est atteinte (l'utilisateur passe alors en liste d'attente), et retirer une inscription avec \$pull. La liste des participants et le pourcentage d'occupation se mettent à jour immédiatement après chaque action.

Campus Event Manager

DashboardEventsUsersAnalytics

Dashboard

15Users

18Events

12Upcoming events

91Total registrations

Next 5 upcoming events

Title	Category	Start date	Location
Student Robotics Meetup	Meetup	12 Sep 2026, 09:00	Engineering Building - Workshop 3
Design Thinking Sprint	Workshop	18 Sep 2026, 09:00	Creative Studio - S2
Intro to Cloud Security	Talk	24 Sep 2026, 09:00	Innovation Center - A102
Open Source Contribution Day	Workshop	03 Oct 2026, 09:00	Tech Hub - Lab 2
Entrepreneurship 101	Talk	08 Oct 2026, 09:00	Main Hall - Auditorium

Most popular event

Student Robotics Meetup

Meetup

9 confirmed registrations

Capacity: 10

View event

Campus Event Manager

DashboardEventsUsersAnalytics

Events

+ New event

Search by title

All categories

All tags

Upcoming & past

Date: soonest first

Filter

Title	Category	Start date	Location	Capacity	Confirmed		
Python for Automation	Workshop	20 Jul 2026, 09:00	Innovation Center - B204	22	3	Past	EditDelete
Career Fair: Tech & Data	Career Fair	27 Jul 2026, 09:00	Main Hall - Auditorium	80	0	Past	EditDelete
Intro to Python for Data Science	Workshop	15 Aug 2026, 09:00	Innovation Center - B204	25	7	Past	EditDelete
Cloud Architecture Fundamentals	Talk	22 Aug 2026, 09:00	Innovation Center - A101	40	3	Past	EditDelete
Campus Hackathon: Smart Mobility	Hackathon	05 Sep 2026, 09:00	Tech Hub - Lab 1	30	0	Past	EditDelete
Cybersecurity Career Night	Career Fair	09 Sep 2026, 09:00	Main Hall - Auditorium	60	7	Past	EditDelete

Campus Event Manager

DashboardEventsUsersAnalytics

Student Robotics Meetup

EditDelete

Meetup

Monthly meetup for the robotics club to share ongoing projects.

robotics

opensource

Start / End

12 Sep 2026, 09:00 → 12 Sep 2026, 11:00

Location

Engineering Building, room Workshop 3 - Paris campus

Organizer

Emma Rousseau

Capacity

9 / 10 (90.0% occupancy)

Register a user

Select a user

Register

Participants (14)

Name	Status	
Alice Martin	Confirmed	Remove
David Nguyen	Waiting	Remove
Lucas Bernard	Waiting	Remove
Sofia Costa	Confirmed	Remove
Fatou Diallo	Confirmed	Remove
Yanis Benali	Waiting	Remove
Marco Rinaldi	Cancelled	Remove
Lea Moreau	Cancelled	Remove

Users

Annuaire des utilisateurs avec recherche par nom/email, filtre par département et par rôle, et le nombre d'inscriptions de chacun, calculé par un pipeline \$unwind + \$group sur la collection events. La suppression d'un utilisateur est bloquée s'il est encore référencé comme organisateur ou dans une inscription, pour ne jamais laisser d'ObjectId orphelin.

Détail d'un utilisateur

Historique complet de participation d'un utilisateur, construit avec un \$lookup vers events dont le sous-pipeline fait un \$unwind sur registrations puis un \$match sur le userId courant - c'est le \$lookup demandé par l'énoncé. Les compteurs «upcoming» et «past» sont recalculés côté Python à partir du même résultat.

Analytics

Les six analyses demandées : inscriptions par catégorie, top 5 des événements les plus populaires, utilisateurs sans aucune inscription, événements au-dessus du taux d'occupation moyen, tags les plus utilisés, et répartition des événements par mois. Le détail des deux pipelines les plus avancés est expliqué dans la section suivante.

Campus Event Manager

DashboardEventsUsersAnalytics

Users

+ New user

Search by name or email

All departments

All roles

Filter

Name	Email	Department	Role	Interests	Registrations
Yanis Benali	yanis.benali@campus.fr	Computer Science	student	cybersecurityopen source	5
Lucas Bernard	lucas.bernard@campus.fr	Mechanical Engineering	student	roboticssports	7
Sofia Costa	sofia.costa@campus.fr	Computer Science	student	open sourcecloud computingvolunteering	7
Fatou Diallo	fatou.diallo@campus.fr	Data Engineering	student	data scienceartificial intelligencesports	7
Chloe Dubois	chloe.dubois@campus.fr	Design	student	designphotography	5
Nora El Amrani	nora.elamrani@campus.fr	Business Administration	student	entrepreneurshipdesign	9
Tom Girard	tom.girard@campus.fr	Computer Science	student	cybersecurityartificial intelligence	9
Hugo Lefevre	hugo.lefevre@campus.fr	Physics	student	roboticsmusic	0
Alice Martin	alice.martin@campus.fr	Data Engineering	student	artificial intelligencecloud computingdata science	7

Campus Event Manager

DashboardEventsUsersAnalytics

Yanis Benali

EditDelete

yanis.benali@campus.fr - Computer Science - student

Interests

cybersecurityopen source

3Upcoming

2Past

Event participation (5)

Event	Category	Date	Status
Intro to Python for Data Science	Workshop	15 Aug 2026, 09:00	Confirmed
Cybersecurity Career Night	Career Fair	09 Sep 2026, 09:00	Confirmed
Student Robotics Meetup	Meetup	12 Sep 2026, 09:00	Waiting
UX Research Workshop	Workshop	04 Nov 2026, 09:00	Cancelled
Security CTF Qualifiers	Hackathon	10 Nov 2026, 09:00	Waiting

Campus Event Manager

DashboardEventsUsersAnalytics

Analytics

A. Registrations by category

Category	Number of events	Confirmed registrations
Workshop	5	23
Meetup	3	15
Talk	5	15
Hackathon	3	8
Career Fair	2	7

B. Top 5 most popular events

Title	Category	Capacity	Confirmed	Occupancy
Student Robotics Meetup	Meetup	10	9	90.0%
Intro to Python for Data Science	Workshop	25	7	28.0%
Cybersecurity Career Night	Career Fair	60	7	11.7%
UX Research Workshop	Workshop	20	6	30.0%
Networking Night with Alumni	Meetup	60	5	8.3%

6. Focus sur deux pipelines d'agrégation

Parmi les six analyses de la page Analytics, deux pipelines méritent une explication un peu plus détaillée parce qu'ils vont au-delà d'un simple group/sort.

Utilisateurs sans aucune inscription (analyse C)

L'idée est de partir de users et d'aller chercher, pour chaque utilisateur, ses éventuelles inscriptions dans events via un \$lookup avec pipeline (nécessaire ici car la correspondance ne se fait pas sur un champ direct mais sur un élément à l'intérieur d'un tableau) :

```
db.users.aggregate([
  { $lookup: { from: "events", let: { uid: "$_id" },
    pipeline: [
      { $unwind: "$registrations" },
      { $match: { $expr: { $eq: ["$registrations.userId", "$$uid"] } } }
    ], as: "regs" } },
  { $match: { regs: { $size: 0 } } }
])
```

Le sous-pipeline du \$lookup éclate d'abord le tableau registrations de chaque événement avec \$unwind, puis ne garde que les lignes où le userId correspond à l'utilisateur courant (\$\$uid, la variable passée par let). Le résultat regs est donc soit vide, soit rempli d'événements où l'utilisateur apparaît. Le \$match final ne garde que les documents où regs est un tableau vide, c'est-à-dire les utilisateurs jamais inscrits nulle part.

Événements au-dessus du taux d'occupation moyen (analyse D)

C'est le pipeline le plus complexe du projet parce qu'il faut calculer une moyenne globale sur tous les événements puis filtrer les événements individuellement par rapport à cette moyenne - deux opérations qui portent sur des granularités différentes dans le même pipeline. J'ai utilisé \$facet pour calculer les deux en parallèle à partir du même jeu de documents :

```
{ $addFields: { confirmedCount: ..., occupancyPct: ... } },
{ $facet: {
  events: [ { $project: {...} } ],
  stats: [ { $group: { _id: null, avgOccupancy: { $avg: "$occupancyPct" } } } ]
} },
{ $unwind: "$stats" },
{ $project: { avgOccupancy: "$stats.avgOccupancy",
  events: { $filter: { input: "$events", as: "e",
    cond: { $gt: ["$$e.occupancyPct", "$stats.avgOccupancy"] } } } } }
```

La branche stats du \$facet calcule la moyenne globale avec \$avg, pendant que la branche events garde chaque événement individuellement. Comme \$facet renvoie un seul document avec les deux branches côte à côte, il suffit ensuite d'un \$filter pour ne garder que les événements dont le pourcentage d'occupation dépasse la moyenne calculée juste à côté. C'est ce pipeline qui utilise concrètement \$avg, une des opérations attendues par l'énoncé.

7. Conclusion

La partie la plus longue n'a pas été l'écriture des pipelines mais la cohérence des données de seed. En testant l'application dans le navigateur, je me suis rendu compte que mon script initial pouvait inscrire deux fois le même utilisateur au même événement quand je complétais artificiellement un événement jusqu'à sa capacité pour tester le badge «Full» - le doublon apparaissait dans la liste des participants. J'ai dû revoir la fonction de génération pour suivre, événement par événement, quels utilisateurs étaient déjà inscrits avant d'en ajouter d'autres. Ça m'a rappelé que la validation côté application (dans register_user, qui vérifie l'absence de doublon avant tout \$addToSet) ne sert à rien si le jeu de données de départ n'est déjà pas propre.

Autre point qui m'a pris du temps : combiner un `$jsonSchema` avec une contrainte inter-champs (`endDate` ne doit pas être avant `startDate`). `$jsonSchema` seul ne sait pas comparer deux champs entre eux, il a fallu envelopper le validateur dans un `$and` avec une expression `$expr` à côté. Une fois compris que les deux mondes (validation de schéma et expressions de requête) peuvent cohabiter dans le même validator, ça a débloqué pas mal de choses.

Comme limite assumée, il n'y a pas d'authentification (hors périmètre du sujet) : n'importe qui peut inscrire ou désinscrire n'importe quel utilisateur depuis la page d'un événement. Dans une vraie version, il faudrait aussi penser à la pagination sur les pages Events et Users si le volume de données grossissait vraiment - avec 15 utilisateurs et 18 événements ça n'a aucun sens de la faire maintenant, mais les index sur `startDate`, `category` et les index composés (`category+startDate`, `department+role`) sont déjà là pour anticiper ce cas. J'ai quand même ajouté une petite suite de tests avec `pytest` (`tests/test_validation.py`) sur les règles de validation des formulaires - pas une couverture complète de l'application, mais assez pour vérifier que les règles métier (capacité, dates, email) ne régressent pas silencieusement si je retouche le code plus tard.